

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Área de desarrollo de software
Software Avanzado

Lic. Marco Tulio Aldana Prillwitz
Xhunik Nikol Miguel Mutzutz



Práctica 3

Índice

1. Resumen	4
2. Objetivos	4
2.1 Objetivo General	4
2.2 Objetivos Específicos	4
3. Enunciado	5
3.1 Definiciones	5
3.2 Arquitectura requerida (Microservicios + Gateway + Frontend + Infra)	5
3.2.1 Frontend (Vite + TypeScript) — obligatorio	5
3.2.2 API Gateway (NestJS, REST) — obligatorio	6
3.2.3 Orders Service (NestJS, gRPC) — obligatorio	6
3.2.4 Pricing Service (NestJS, gRPC) — obligatorio	6
3.2.5 Receipt Service (NestJS, gRPC) — obligatorio	6
3.2.6 FX Service (externo + fallback + Redis) — obligatorio	6
3.2.7 Infraestructura obligatoria en GKE	7
3.3 Diagrama (obligatorio en README)	7
3.4 Persistencia obligatoria (MSSQL)	7
3.5 Caché obligatoria (Redis)	7
3.6 Exposición a Internet: Ingress → Gateway → interno (obligatorio)	8
3.7 Kubernetes en GKE (manifiestos, probes, rollout/rollback)	8
3.7.1 Manifiestos mínimos obligatorios	8
3.7.2 Probes obligatorias	8
3.7.3 docker-compose.yml (obligatorio)	8
3.8 CI/CD end-to-end (obligatorio)	8
3.9 Versionado de imágenes en registry (tagging obligatorio, sin ambigüedad)	9
3.10 Resiliencia en integraciones externas (timeouts/retries/backoff/circuit breaker)	9
3.11 Observabilidad de logs: ELK completo + Grafana (obligatorio)	9
3.11.1 ELK completo obligatorio en GKE	9
3.11.2 Agente de recolección (libre) → Logstash (obligatorio)	10
3.11.3 Cobertura obligatoria de logs	10
3.11.4 Grafana obligatorio consultando Elasticsearch	10
3.12 Pruebas no funcionales: smoke + Locust en GKE (obligatorio)	10
3.12.1 Smoke test post-deploy (CI)	10
3.12.2 Locust (obligatorio)	10
4. Entregables	11
4.1 Evidencias obligatorias	11
5. Requerimientos mínimos	12
6. Restricciones	12
7. Consideraciones adicionales	13
7.1 Historial de desarrollo	13
7.2 Integridad académica	13
8. Fecha y Modo de entrega	13

1. Resumen

En esta práctica extenderás el sistema **Quetzal Ship** para operar un **ciclo de entrega tipo producción** desplegado **completamente en Google Kubernetes Engine (GKE)**.

El objetivo principal es evidenciar dominio de:

- **CI/CD end-to-end** (Build → Test → Post-Build → Release/Delivery → Deploy)
- **Imágenes versionadas** publicadas en un registry
- **Kubernetes operativo** en GKE con Ingress, probes y rollout/rollback
- **Integraciones externas resilientes** (timeouts/retries/backoff/circuit breaker)
- **Servicio FX con API alternativa + caché obligatoria en Redis**
- **Observabilidad de logs** con **ELK completo** y visualización/búsqueda en **Grafana** (sin Prometheus ni Node Exporter en esta etapa)
- **Prueba de carga con Locust** ejecutada dentro del cluster y contra el endpoint público (Ingress)

Regla principal: todos los recursos (aplicación, datos, observabilidad, carga) deben estar en el **cluster de GKE**.

2. Objetivos

2.1 Objetivo General

Construir y operar un sistema distribuido (microservicios) para Quetzal Ship, desplegado en **GKE**, verificable mediante un **pipeline CI/CD reproducible**, con persistencia, resiliencia, logs centralizados y pruebas no funcionales.

2.2 Objetivos Específicos

1. Desplegar en Kubernetes y operar el ciclo de entrega desde CI, cumpliendo entregables mínimos (manifiestos, probes, rollout/rollback, pipeline y evidencias).
2. Implementar un pipeline completo (Build/Test/Post-Build/Release/Delivery/Deploy) y evidenciarlo con pipelines verdes.
3. Publicar imágenes versionadas en un registry con reglas de tagging deterministas.
4. Tener un despliegue Kubernetes operativo en GKE (Ingress, Services internos, ConfigMaps/Secrets, probes, despliegue reproducible).
5. Implementar integraciones externas resilientes (timeouts, retries, backoff y circuit breaker) con degradación elegante.
6. Implementar un servicio FX con proveedor principal + proveedor alternativo y caché en Redis.
7. Centralizar los logs con ELK completo y consultarlos desde Grafana con un dashboard mínimo.
8. Ejecutar una prueba de carga con Locust dentro de GKE contra el endpoint público, simulando un flujo tipo producción.

3. Enunciado

3.1 Definiciones

Término	Definición
Zona	METRO, INTERIOR, FRONTERA
Tipo de servicio	STANDARD, EXPRESS, SAME_DAY
Estado	ACTIVE, CANCELLED
Orden	Conjunto de paquetes con origen, destino y tipo de servicio común, más descuentos/opciones
Paquete	Unidad física enviada dentro de una orden
Desglose	Valores intermedios del cálculo: pesos, subtotales, recargos, descuento y total

3.2 Arquitectura requerida (Microservicios + Gateway + Frontend + Infra)

Debes implementar componentes en ejecución, cada uno en su propio contenedor, desplegados en GKE.

3.2.1 Frontend (Vite + TypeScript) — obligatorio

- SPA creada con Vite + TypeScript.
- Debe permitir ejecutar los flujos mínimos del sistema (crear/consultar/listar/cancelar, generar recibo, conversión de moneda al mostrar total/recibo).
- Restricción de acceso: el Frontend **no debe exponer acceso directo a microservicios internos**; todo consumo debe ser vía Gateway.

3.2.2 API Gateway (NestJS, REST) — obligatorio

El Gateway es el **único punto de entrada del sistema Quetzal Ship** desde Internet.

Responsabilidades mínimas:

- Exponer endpoints REST (documentados en OpenAPI).
- Validar entradas (payload y reglas básicas).
- Orquestar llamadas internas a los microservicios.
- Implementar y propagar **correlationId/requestId** hacia todos los servicios para trazabilidad en logs.
- Mapear errores internos a códigos HTTP coherentes (ver 3.9 y 3.10).

3.2.3 Orders Service (NestJS, gRPC) — obligatorio

Responsabilidades mínimas:

- Crear, consultar, listar y cancelar órdenes.
- Persistir órdenes y paquetes en **MSSQL** (ver 3.3).
- Al crear orden, invocar a Pricing para calcular y guardar desglose/total (para recibo estable).

3.2.4 Pricing Service (NestJS, gRPC) — obligatorio

Responsabilidades mínimas:

- Implementar exactamente las **mismas reglas de cálculo** de la Práctica 2 (no se redefinen aquí).
- Exponer un endpoint interno (gRPC/HTTP según su arquitectura) para calcular desglose/total.
- Debe estar cubierto por pruebas unitarias.

3.2.5 Receipt Service (NestJS, gRPC) — obligatorio

Responsabilidades mínimas:

- Generar recibo para una orden existente.
- El recibo debe incluir: datos de la orden, desglose, total, moneda, y si aplica conversión.
- Debe usar datos persistentes (no recalcular “a ciegas” sin consistencia).

3.2.6 FX Service (externo + fallback + Redis) — obligatorio

Responsabilidades mínimas:

- Proveer tipo de cambio (base/quote) y/o conversión de montos.
- Consumir un **proveedor externo A** y un **proveedor externo B alternativo** (fallback).
- Implementar **caché obligatorio con Redis** (TTL configurable).
- Implementar resiliencia en llamadas externas (ver 3.9).

3.2.7 Infraestructura obligatoria en GKE

Dentro del cluster debe existir y estar operativa:

- **MSSQL** con persistencia (PVC).
- **Redis** para caché FX.
- **ELK completo**: Elasticsearch + Logstash + Kibana.
- **Grafana** (para consultas de logs en Elasticsearch).
- **Locust** (para pruebas de carga).

3.3 Diagrama (obligatorio en README)

En el README debe existir un diagrama de alto nivel que muestre:

- Entrada pública: **Internet** → **Ingress** → **API Gateway**.
- Comunicación interna hacia microservicios.
- Dependencias: MSSQL, Redis, ELK, Grafana, Locust.
- Flujo de logs: Agente → Logstash → Elasticsearch → Kibana.
- Flujo de logs: Grafana.

3.4 Persistencia obligatoria (MSSQL)

- La base de datos debe ser **MS SQL Server** y debe ejecutarse dentro de GKE.
- Debe ser **persistente** mediante PVC.
- Debe existir un mecanismo reproducible de creación/actualización de esquema (migraciones o scripts automatizados).
- Conexiones y credenciales:
 - Configuradas por envvars.
 - Guardadas en **Secrets** (no repositorio).

Mínimo de datos a persistir:

- Orden (estado, origen, destino, tipo, timestamps y campos de negocio relevantes).
- Paquetes asociados.
- Desglose/total calculado (para recibo estable y auditable).

3.5 Caché obligatoria (Redis)

- Redis debe ejecutarse dentro de GKE.
- Se usará como caché del FX Service con TTL configurable.
- La clave debe permitir consultar por par base/quote (ej. **GTQ:USD**).
- Debe permitir modo degradación usando última tasa cacheada si la API externa falla.

3.6 Exposición a Internet: Ingress → Gateway → interno (obligatorio)

- Todos los accesos desde fuera deben pasar por **Ingress** y deben llegar primero al **API Gateway**.
- No se permite exposición pública directa de microservicios internos (Orders/Pricing/Receipt/FX) ni de bases de datos (MSSQL/Redis) ni del pipeline de logs (Logstash/Elasticsearch).
- Grafana y Kibana pueden exponerse por Ingress (host separado recomendado), pero deben estar protegidos.

3.7 Kubernetes en GKE (manifiestos, probes, rollout/rollback)

Objetivo: desplegar en Kubernetes y operar el ciclo de entrega desde CI.

3.7.1 Manifiestos mínimos obligatorios

Debes entregar manifiestos Kubernetes para el sistema completo, incluyendo al menos:

- **Deployment** (por componente).
- **Service** (por componente).
- **Ingress** (para exposición a Internet).
- **ConfigMap** (config not sensible).
- **Secret** (credenciales, tokens, strings sensibles).

3.7.2 Probes obligatorias

- **readinessProbe** y **livenessProbe** al menos para: Gateway, Orders, FX (y recomendado para el resto).
- Las probes deben reflejar disponibilidad real (no “siempre OK”).

3.7.3 docker-compose.yml (obligatorio)

En el README debes documentar:

- Cómo desplegar una nueva versión (rollout).
- Cómo revertir a una versión anterior (rollback).
- Evidencia de comandos usados en GKE (ver sección 4 y 5).

3.8 CI/CD end-to-end (obligatorio)

Debes implementar un pipeline en CI que incluya como mínimo:

1. **Build:** compiler backend + frontend.
2. **Test:** ejecutar unit tests y validación de contratos (OpenAPI y contratos internos).
3. **Post-Build:** build de imágenes Docker.
4. **Release/Delivery:** push de imágenes al registry con tags deterministas.

5. **Deploy:** despliegue automático al cluster de GKE.
6. **Post-Deploy:**
 - Smoke test (peticiones HTTP al endpoint público del Ingress).
 - Ejecución de Locust dentro del cluster (ver 3.11).

3.9 Versionado de imágenes en registry (tagging obligatorio, sin ambigüedad)

- Todas las imágenes deben publicarse en un registry accesible por el cluster.
- Deben existir reglas de tagging deterministas, por ejemplo:
 - `main-<short_sha>` para builds en main.
 - `vX.Y.Z` para releases con tag Git.
- Debe existir evidencia de tags por servicio.

3.10 Resiliencia en integraciones externas (timeouts/retries/backoff/circuit breaker)

Obligatorio para FX (y recomendado para llamadas internas si aplica):

- Timeout configurable por variables de entorno.
- Retries limitados y solo para errores transitorios.
- Backoff (exponencial recomendado).
- Circuit breaker con umbrales documentados (apertura/cierre).

Degradación elegante obligatoria (FX):

- Si el proveedor A falla, usar proveedor B.
- Si ambos fallan, usar la última tasa de Redis si existe (`stale=true`).
- Si no existe tasa cacheada, responder error controlado (sin colapsar el sistema).

3.11 Observabilidad de logs: ELK completo + Grafana (obligatorio)

En esta etapa:

- **No se usa Prometheus ni Node Exporter.**
- El enfoque es **logs centralizados** y un dashboard mínimo.

3.11.1 ELK completo obligatorio en GKE

Debe estar desplegado y operativo:

- Elasticsearch
- Logstash
- Kibana

3.11.2 Agente de recolección (libre) → Logstash (obligatorio)

- Puedes elegir el agente (Fluent Bit / Fluentd / Filebeat / otro).
- Condición: el agente debe enviar los logs **a Logstash** (no directo a Elasticsearch).
- Logstash debe indexar en Elasticsearch.

3.11.3 Cobertura obligatoria de logs

Deben capturarse y consultarse logs como mínimo de:

- Ingress Controller
- API Gateway
- Orders
- Pricing
- Receipt
- FX
- Frontend (si aplica)

3.11.4 Grafana obligatorio consultando Elasticsearch

- Grafana debe ejecutarse en GKE.
- Debe tener datasource de Elasticsearch.
- Debe existir un dashboard mínimo con:
 - Filtro por servicio.
 - Filtro por nivel (**info/warn/error** o equivalente).
 - Búsqueda por **correlationId/requestId**.
 - Vista de errores agregada (por ejemplo conteo de logs de error por intervalo).

3.12 Pruebas no funcionales: smoke + Locust en GKE (obligatorio)

3.12.1 Smoke test post-deploy (CI)

- Tras desplegar, el pipeline debe ejecutar un smoke test contra el endpoint público del Ingress (Gateway).
- Debe validar, al menos:
 - **health/status** del Gateway
 - Un flujo mínimo (crear → consultar → recibo) o equivalente.

3.12.2 Locust (obligatorio)

- Locust debe ejecutarse **dentro de GKE** (Job/Deployment).
- Debe atacar el endpoint público:
Locust → Ingress → API Gateway → sistema interno
- Debe simular un flujo tipo producción:
 - Crear orden → consultar orden → obtener recibo
 - Un porcentaje de requests debe ejercitar conversión de moneda (FX)

4. Entregables

Se entrega un repositorio por práctica en GitHub con nombre:

- `<NUM_GRUPO>_SoftwareAvanzado_Practica3`.

Además deben agregar el usuario del auxiliar como colaborador:

- **xhuniktzi**

Archivos y estructura mínima en el repositorio:

- `README.md`: guía de ejecución, despliegue y verificación, incluyendo diagrama y rollback/rollout.
- `INTEGRANTES.md`: 5 integrantes: nombre/carné/rol.
- Manifiestos Kubernetes (carpeta `k8s/` o equivalente) para:
 - App (Gateway, Orders, Pricing, Receipt, FX, Frontend si aplica)
 - MSSQL + PVC
 - Redis
 - ELK (E+L+K)
 - Agente de logs → Logstash
 - Grafana
 - Locust
 - Ingress(es)
- Pipeline CI/CD (GitHub Actions) con stages end-to-end y despliegue a GKE. Planning
- Evidencias en `docs/evidence/` (ver sección 4.1)

4.1 Evidencias obligatorias

Debes incluir capturas o salidas pegadas (ordenadas y con títulos) de:

A. Kubernetes (GKE)

- `kubectl get pods,svc,ingress -n <namespace>`
- `kubectl describe ingress <name> -n <namespace>`
- Evidencia de probes (describe de Deployment/Pod).
- Evidencia/documentación de rollout y rollback.

B. CI/CD

- Capturas/enlaces de pipelines verdes (build/test, push imágenes, deploy, post-deploy, locust).

C. Registry

- Evidencia de tags versionados por imagen/servicio.

D. Endpoints activos

- Evidencia de endpoints funcionando a través del Ingress (curl o similar).

E. Logs (ELK + Grafana)

- Evidencia de índices en Elasticsearch y logs por servicio.
- Evidencia de ingesta vía Logstash.
- Captura del dashboard en Grafana consultando Elasticsearch.

F. Locust

- Capturas del resultado (RPS, latencias, errores) y/o reporte exportado.

5. Requerimientos mínimos

Para tener derecho a calificación, debe cumplirse todo lo siguiente (si falta uno, no se califica):

1. Todo el sistema (app + infra + observabilidad + carga) está desplegado en **GKE**.
2. Existe **Ingress** y el flujo externo cumple: **Internet** → **Ingress** → **API Gateway** → **interno**.
3. Persistencia con **MSSQL** (PVC) operativa y usada por Orders/Receipt.
4. Caché con **Redis** usada por FX obligatoriamente.
5. Pipeline CI/CD end-to-end: build/test/push/deploy/post-deploy.
6. Imágenes versionadas en registry con tags deterministas por servicio.
7. Manifiestos mínimos: Deployment/Service/Ingress/ConfigMap/Secret.
8. Probes configuradas + rollout/rollback documentado.
9. FX con proveedor alternativo + resiliencia (timeouts/retries/backoff/circuit breaker) + degradación elegante.
10. **ELK completo** operativo y logs de todos los servicios llegan a Elasticsearch **vía Logstash**.
11. Grafana consultando logs desde Elasticsearch con dashboard mínimo.
12. Locust ejecutado en GKE atacando el endpoint del Ingress, con evidencias.

6. Restricciones

- Todo debe ejecutarse en GKE (no evidencia final fuera del cluster).
- Todo acceso desde fuera debe pasar por Ingress y luego por API Gateway (no accesos directos a microservicios internos).
- Observabilidad en esta etapa: logs con ELK + Grafana; **no Prometheus, no Node Exporter**.
- ELK completo obligatorio (E+L+K).
- Agente de recolección libre, pero debe enviar logs a Logstash.
- Secrets fuera del repositorio (credenciales y tokens en Secrets).
- No se acepta “microservicios por capas” (la separación debe ser por responsabilidad de negocio).

7. Consideraciones adicionales

7.1 Historial de desarrollo

Se revisará el historial de commits para verificar avance incremental (no realizado en un único commit).

7.2 Integridad académica

La copia total o parcial implica calificación de 0 y será reportada a la escuela de ciencias y sistemas.

8. Fecha y Modo de entrega

- **Fecha límite:** domingo 28 de diciembre, 23:59.
- **Entrega:** enlace del repositorio GitHub en la plataforma UEDi. El repositorio debe ser accesible e incluir todos los entregables y tags requeridos.